TECHNICAL DOCUMENTATION

“RAG-Based College/University FAQ Assistant using LangGraph with Human-in-the-Loop (HITL)”

Introduction

Retrieval-Augmented Generation (RAG) is an advanced approach that combines information retrieval with natural language generation. It enhances the capability of Large Language Models (LLMs) by providing relevant external knowledge during response generation.

Traditional language models rely only on pre-trained data, which limits their ability to provide accurate and up-to-date information. RAG overcomes this limitation by retrieving relevant data from a knowledge base and using it to generate context-aware responses.

In this project, a RAG-based Customer Support Assistant is developed using a PDF knowledge base. The system is designed to answer user queries related to college policies such as attendance, exams, grading, and fees.

Additionally, the system incorporates LangGraph for workflow orchestration and includes a Human-in-the-Loop (HITL) mechanism to handle complex or uncertain queries. This ensures improved reliability, accuracy, and user trust.

This system demonstrates how modern AI techniques can be combined with structured workflows to build intelligent and reliable applications for real-world use cases.

System Architecture Explanation

The system is designed using a modular architecture that integrates document processing, vector-based retrieval, and language model generation.

The process begins with loading a PDF document, which acts as the knowledge base. The document is processed and divided into smaller chunks, which are then converted into embeddings using an embedding model. These embeddings are stored in a vector database (ChromaDB).

When a user submits a query, the system retrieves the most relevant chunks from the database using similarity search. These retrieved chunks are passed as context to the Large Language Model (LLM), which generates a response.

LangGraph is used to manage the workflow of the system. It controls the flow of data between different components and enables conditional routing. Based on the confidence of the generated response, the system decides whether to return the answer directly or escalate it to a human using the HITL module.

This architecture ensures efficient retrieval, accurate response generation, and reliable handling of complex queries.

The modular design ensures that each component can be independently improved or scaled without affecting the overall system.

Design Decisions

1. Chunk Size Selection
   A moderate chunk size is chosen to balance context completeness and retrieval efficiency. Smaller chunks improve search accuracy, while larger chunks provide better context.
2. Embedding Strategy
   HuggingFace embedding models are used due to their strong semantic representation capabilities and ease of integration.
3. Retrieval Approach
   Similarity-based retrieval is used to fetch the most relevant chunks from the vector database. This ensures that the LLM receives meaningful context.
4. Vector Database Choice
   ChromaDB is selected because it is lightweight, easy to use, and optimized for embedding storage and retrieval.
5. Prompt Design
   The prompt is designed to include both the user query and retrieved context, ensuring that the LLM generates accurate and context-aware responses.
6. Workflow Management
   LangGraph is used instead of linear pipelines to enable flexible and conditional workflows.

Workflow Explanation

The system uses LangGraph to manage the execution flow through a structured graph-based approach. The workflow consists of multiple nodes connected through edges, with a shared state object that carries information between them.

The process begins at the input node, where the user query is received. The query is then passed to the processing node, which performs retrieval and response generation. In this step, the system retrieves relevant chunks from the vector database and provides them as context to the LLM.

The LLM generates a response based on both the query and retrieved context. This response, along with the confidence score, is stored in the state object.

The routing logic then evaluates the response. If the confidence level is sufficient, the system proceeds to the output node, which returns the response to the user. If the confidence is low, the workflow redirects the query to the HITL module for human intervention.

This graph-based workflow ensures flexibility, scalability, and better control over the execution process.

This workflow design allows dynamic decision-making and improves system reliability compared to linear pipelines.

Conditional Logic

The system uses conditional logic to determine how each query should be handled. After generating a response, the system evaluates its quality using a confidence score and relevance of retrieved context.

If the retrieved information is relevant and the generated response is clear, the system directly returns the answer to the user. This represents the automatic response mode.

However, if the system detects low confidence, missing context, or ambiguity in the query, it triggers escalation. In such cases, the query is routed to the Human-in-the-Loop (HITL) module for further handling.

This conditional routing ensures that the system maintains a balance between automation and reliability, reducing incorrect responses while improving user trust.

This mechanism enables intelligent decision-making and prevents the system from providing misleading information.

Human-in-the-Loop (HITL) Implementation

The Human-in-the-Loop (HITL) mechanism is incorporated to handle cases where the system is unable to provide a reliable or confident response. It serves as a critical fallback layer to ensure accuracy, reliability, and user trust.

When a query is processed, the system evaluates the generated response based on factors such as confidence score, relevance of retrieved context, and completeness of the answer. If the system detects low confidence, insufficient context, or ambiguity in the query, it triggers the HITL module.

Upon escalation, the query is forwarded to a human operator or domain expert. The human reviews the query, analyzes the available context, and provides an appropriate and validated response. This response is then treated as the final output and delivered to the user.

Additionally, the HITL mechanism can be extended to log such queries for future improvements, enabling the system to learn from human feedback and enhance its performance over time.

The HITL mechanism enhances the robustness of the system by combining automated intelligence with human judgment. Although it may introduce slight delays in response time, it significantly improves reliability, reduces the risk of incorrect answers, and ensures better handling of edge cases.

Challenges and Trade-offs

1. Retrieval Accuracy vs Speed
   Improving retrieval accuracy often requires more complex embedding models and larger chunk sizes, which can increase processing time. A balance must be maintained between accuracy and system responsiveness.
2. Chunk Size vs Context Quality
   Smaller chunks improve retrieval precision but may lose important context. Larger chunks provide better context but may reduce retrieval efficiency. Selecting an optimal chunk size is a key design decision.
3. Cost vs Performance
   Using high-performance models improves response quality but increases computational cost. Lightweight models are more efficient but may reduce accuracy.
4. Automation vs Reliability
   Fully automated systems provide faster responses but may produce incorrect answers. Introducing HITL improves reliability but adds latency.
5. Scalability vs Complexity
   Scaling the system to handle large datasets and multiple users increases system complexity. Efficient design is required to maintain performance.
6. Latency vs User Experience
   Reducing response time improves user experience, but aggressive optimization may affect the quality of responses.

These trade-offs were carefully considered to design a balanced system that performs efficiently while maintaining accuracy and reliability.\

Testing Strategy

The system is tested to ensure accurate retrieval, correct response generation, and proper workflow execution.

1. Functional Testing
   The system is tested with various user queries to verify that it correctly retrieves relevant information and generates appropriate responses.
2. Retrieval Testing
   Queries are designed to check whether the retriever fetches the most relevant chunks from the vector database.
3. Response Validation
   Generated responses are evaluated for correctness, clarity, and relevance to the user query.
4. Routing Logic Testing
   The conditional routing mechanism is tested to ensure that queries with low confidence are correctly escalated to the HITL module.
5. Edge Case Testing
   Special cases such as empty queries, ambiguous questions, and missing data are tested to ensure the system handles them gracefully.
6. HITL Testing
   The HITL workflow is tested by simulating low-confidence scenarios and verifying that human responses are correctly integrated into the system.

Future Enhancements

1. Multi-Document Support
   The system can be extended to handle multiple PDF documents, enabling a broader knowledge base and more comprehensive responses.
2. Feedback Learning Mechanism
   User and human feedback can be stored and used to improve retrieval accuracy and response generation over time.
3. Memory Integration
   Adding conversational memory will allow the system to maintain context across multiple queries, improving user interaction.
4. Advanced LLM Integration
   More powerful or fine-tuned language models can be integrated to enhance response quality and understanding.
5. Real-Time Deployment
   The system can be deployed as a web application using cloud platforms such as AWS or Firebase for real-world usage.
6. Improved HITL System
   The HITL module can be enhanced with real-time human support, dashboards, and ticketing systems for better management.